Optimizing Spark Jobs

Formats, Memory, and Execution Strategies

Dano Lee

Two Axes of Apache Spark Optimization

- **Compute efficiency within each task**
- Data movement across the cluster

Optimizing Spark Jobs

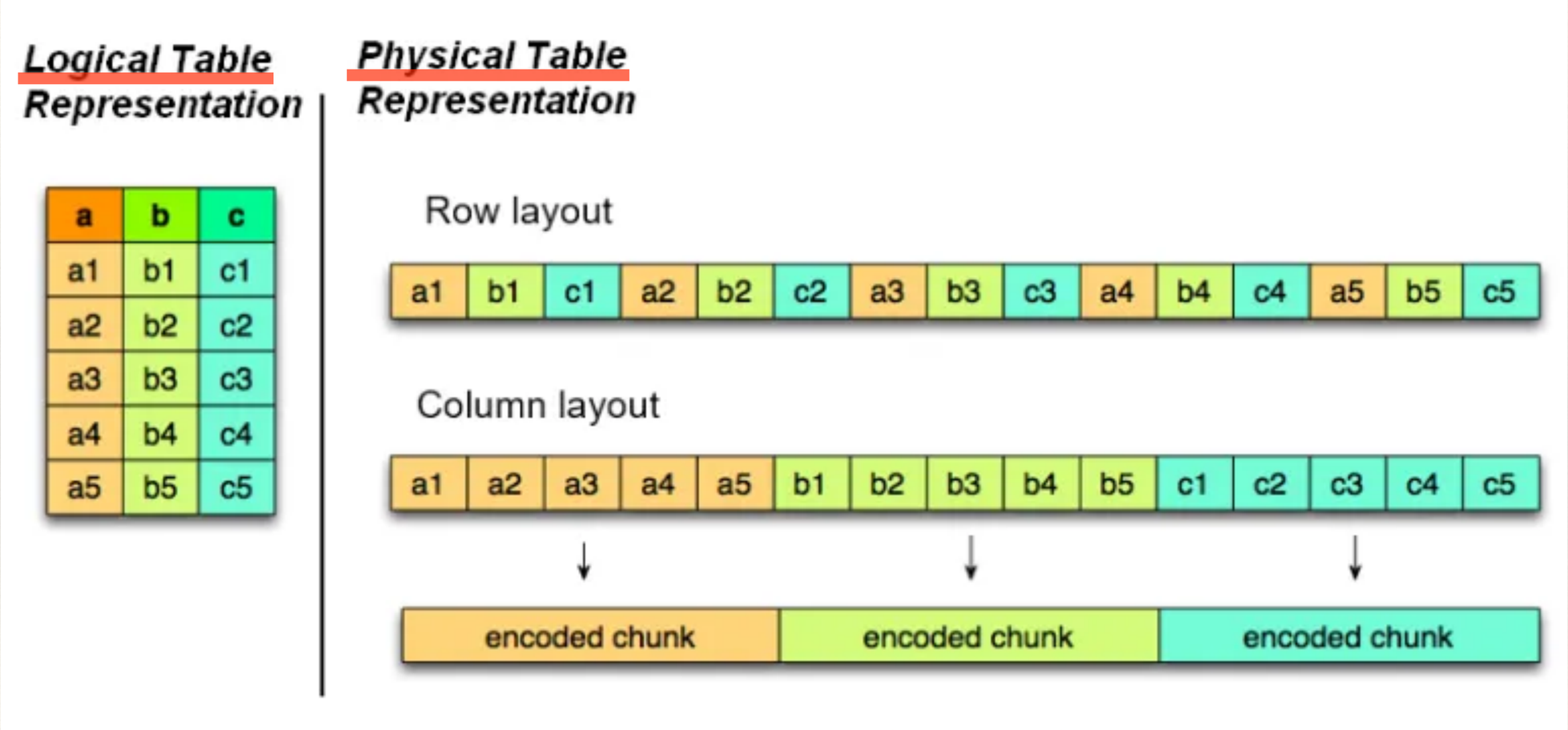
When Should You Use RDDs Instead of DataFrames?

- Because of their higher-level API and optimizations, DataFrames are typically easier to use and offer better performance;
- however, due to their lower-level nature, RDDs can still be useful for defining custom operations, as well as debugging complex data processing tasks.
- RDDs offer **more granular control over partitioning and memory usage.**
- **When dealing with raw, unstructured data, such as text streams, binary files, or custom formats**, RDDs can be more flexible, allowing for custom parsing and manipulation in the absence of a predefined structure.

Use an efficient file format and compression codec

- The file format and compression codec that you use can have a significant impact on I/O and overall Spark performance.
- Using a file format that is optimized for Spark, such as **Apache Parquet** or Apache ORC, with a fast compression codec such as Snappy or LZ4 can reduce the amount of data read and written and improve the performance of your Spark applications.

Row vs Column



source

Advantages of Columnar Storage Over Row-Based Storage:

- **Compression Efficiency**: Columnar storage allows for better compression techniques. Since column values are often of the same type, compression algorithms can be tailored to that type, resulting in higher compression ratios. This reduces storage requirements and speeds up data transfers.
- **Column Pruning**: When executing queries, columnar databases can skip irrelevant columns, reducing I/O and improving query performance. In row-based storage, entire rows must be read, even if only a few columns are needed.
- **Aggregation Performance**: Columnar storage is highly efficient for aggregate queries, as aggregations involve operations on single columns. This leads to faster query performance for analytics tasks.
- **Predicate Pushdown**: Columnar databases can apply filters early in the query execution process by analyzing metadata, minimizing the amount of data read from storage. This feature significantly speeds up query processing.
- **Schema Evolution**: Columnar storage formats like Parquet support schema evolution, enabling the addition of new columns or changes to existing columns without disrupting existing data.

Relational DB vs Dataware House

	Amazon RDS	Amazon Redshift
Purpose	OLTP (Transactional)	OLAP (Analytics)
Data Structure	Row-based storage	Columnar storage
Query Performance	Fast for small transactions	Optimized for large-scale analytics
Scaling	Vertical + Read Replicas	Horizontal (MPP architecture)
Use Case	Applications, websites, CRM, ERP	Business intelligence, data analysis, reporting
Database Engines	MySQL, PostgreSQL, Oracle, SQL Server, Aurora	Amazon Redshift (based on PostgreSQL)
Optimized For	Real-time transactions	Batch processing and analytics
Data Size	Gigabytes to terabytes	Terabytes to petabytes

Data Warehouse vs Data Lake vs Lake House

Data Warehouse

- “A data warehouse is a central repository of information that can be analyzed to make more informed decisions.”
- “Data flows into a data warehouse from transactional systems, relational databases, and other sources, typically on a regular cadence. Business analysts, data engineers, data scientists, and decision makers access the data through business intelligence (BI) tools, SQL clients, and other analytics applications.”

source

Data Lake

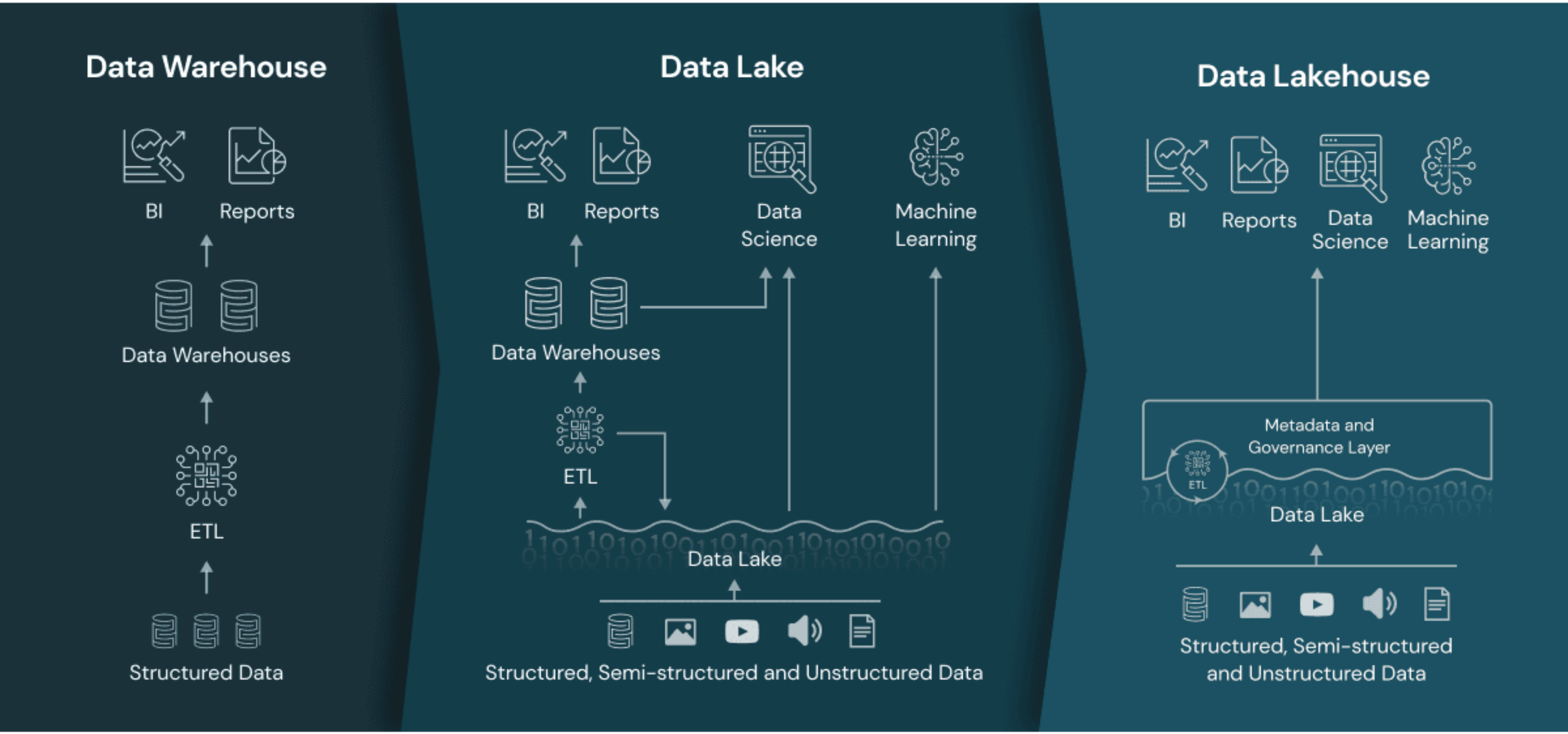
- “A data lake is a centralized repository that allows you to store all your structured and unstructured data at any scale.”
- “You can store your data as-is, **without having to first structure the data**, and run different types of analytics—from dashboards and visualizations to big data processing, real-time analytics, and machine learning to guide better decisions.”

source

Data Lakehouse

- “The data lakehouse is an open data management architecture that combines the flexibility, cost-efficiency, and scale of data lakes with the data management and ACID transactions of data warehouses.”
- A lakehouse is a new, open architecture that combines the best elements of data lakes and data warehouses. Lakehouses are enabled by a new system design: **implementing similar data structures and data management features to those in a data warehouse directly on top of low cost cloud storage in open formats.**
- They are what you would get if you had to redesign data warehouses in the modern world, now that cheap and highly reliable storage (in the form of object stores) are available.

source


Data Warehouse
BI
Reports
Data Warehouses
ETL
Structured Data
Data Lake
BI
Reports
Data Science
Machine Learning
Data Warehouses
ETL
Data Lake
Structured, Semi-structured and Unstructured Data
Data Lakehouse
BI
Reports
Data Science
Machine Learning
Metadata and Governance Layer
ETL
Data Lake
Structured, Semi-structured and Unstructured Data

source

Aspect	Data Warehouse	Data Lake	Lakehouse
Primary Purpose	Central repository for **structured** data, optimized for **BI** and **OLAP** queries	Store **all data types** (structured, semi-structured, unstructured) in **raw or lightly processed** format	Combine Data Warehouse performance/governance with Data Lake flexibility/scalability
Data Structure & Schema	**Schema-on-write** (predefined schema)	**Schema-on-read** (applied at analysis time)	Typically **schema-on-read + ACID transactions** (via table formats like Delta, Hudi, Iceberg)
Types of Data	Primarily **structured** data	**Structured, semi-structured, and unstructured** (text, images, logs, etc.)	Same range as Data Lake, but supports **warehouse-like** query performance and transactional consistency
Data Governance & Quality	Well-governed; **clean, curated** data with rigorous ETL	Governance can be **less formal**; easy to become a "data swamp" without proper controls	Emphasizes **governance** (ACID, versioning, lineage) over raw or curated data in a single platform
Analytics Performance	High-performance for **aggregations** and **reporting**	Relies on big data engines (e.g., Spark, Presto) for large-scale analytics; performance can vary due to raw formats	Designed for near **warehouse-level** performance with advanced features (caching, indexing, query acceleration)
Cost & Scalability	Typically **higher cost** at scale; compute & storage may be tightly coupled	Generally **lower storage cost** (cloud object stores), pay-as-you-go compute	Seeks to maintain low-cost object storage plus on-demand compute scaling, bridging data lake & warehouse
Representative Vendors	Redshift, Snowflake, BigQuery	S3, ADLS, GCS with a data catalog	Object storage + Iceberg, Delta Lake, or Hudi + a query engine
Approx. Emergence	**1980s –** Concepts credited to Barry Devlin & Bill Inmon ("Father of Data Warehousing") Widespread adoption grew in the 1990s with OLAP & BI needs	**Early 2010s –** Term "Data Lake" popularized by James Dixon (Pentaho) Became mainstream with cloud object storage (AWS S3, etc.)	**Late 2010s –** Concept introduced/popularized by Databricks ("Lakehouse Architecture") Rapid evolution via Delta Lake, Apache Hudi, Apache Iceberg, etc.

Apache Parquet vs Other Formats

- Parquet is optimized for the Write Once Read Many (WORM) paradigm. It's slow to write, but incredibly fast to read, **especially when you're only accessing a subset of the total columns**.
- For workloads that frequently process or exchange complete records, a row-oriented format such as **Avro** may be more appropriate. CSV and JSON are generally chosen for interoperability and readability rather than performance.

Apache Parquet vs Other Formats

Spark Format Showdown		File Format		
		CSV	JSON	Parquet
Attribute	Columnar	No	No	**Yes**
	Compressable	**Yes**	**Yes**	**Yes**
	Splittable	**Yes***	**Yes****	**Yes**
	Human Readable	**Yes**	**Yes**	No
	Nestable	No	**Yes**	**Yes**
	Complex Data Structures	No	**Yes**	**Yes**
	Default Schema: Named columns	Manual	**Automatic (full read)**	**Automatic (instant)**
	Default Schema: Data Types	Manual (full read)	**Automatic (full read)**	**Automatic (instant)**

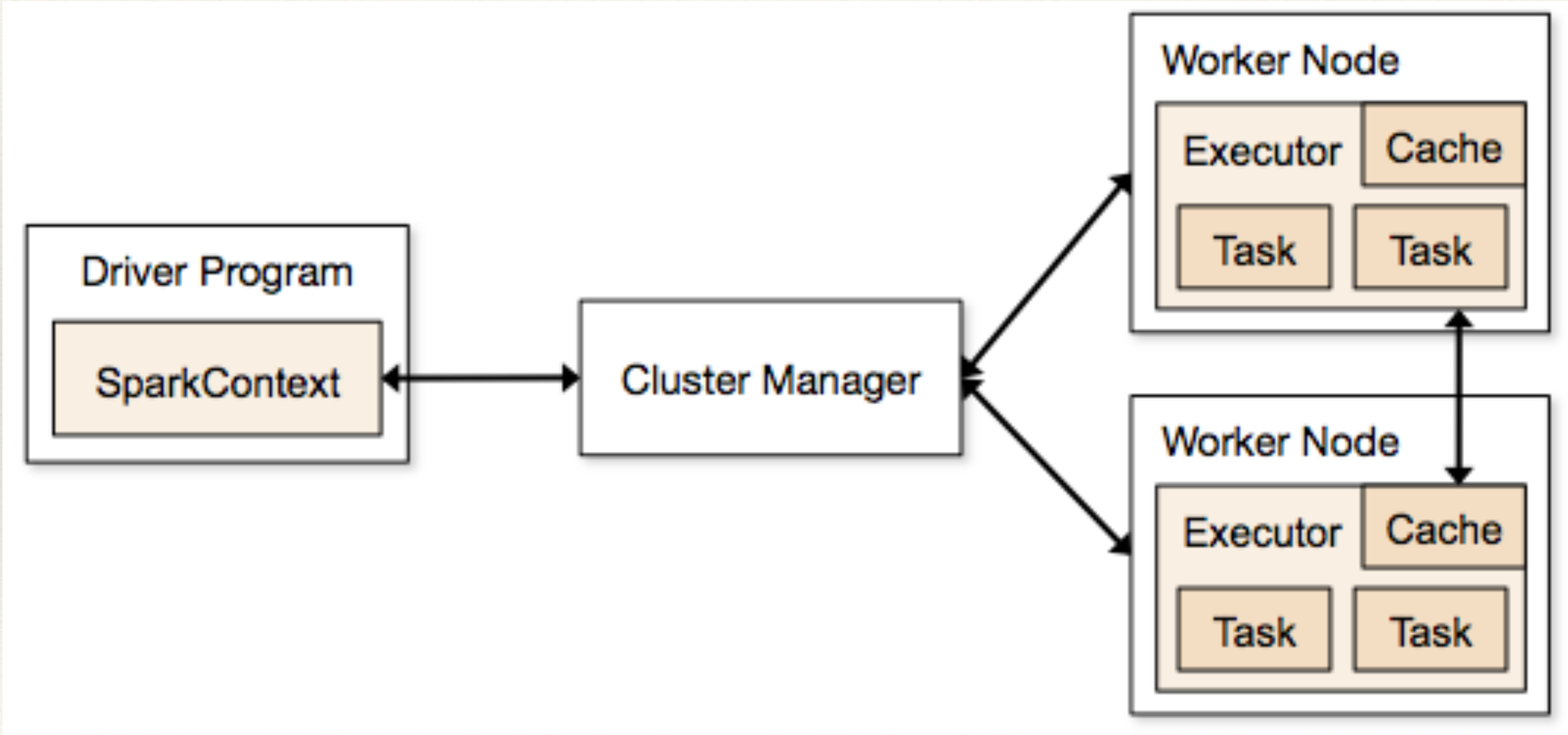
source

Apache Spark and JVM

- Spark is predominantly in Scala and runs on **Java virtual machines** (JVMs)
- PySpark additionally uses Python worker processes when Python code must be executed.

Spark is not a memory-only processing engine.

It uses memory as a working area and manages persisted blocks as an LRU cache.

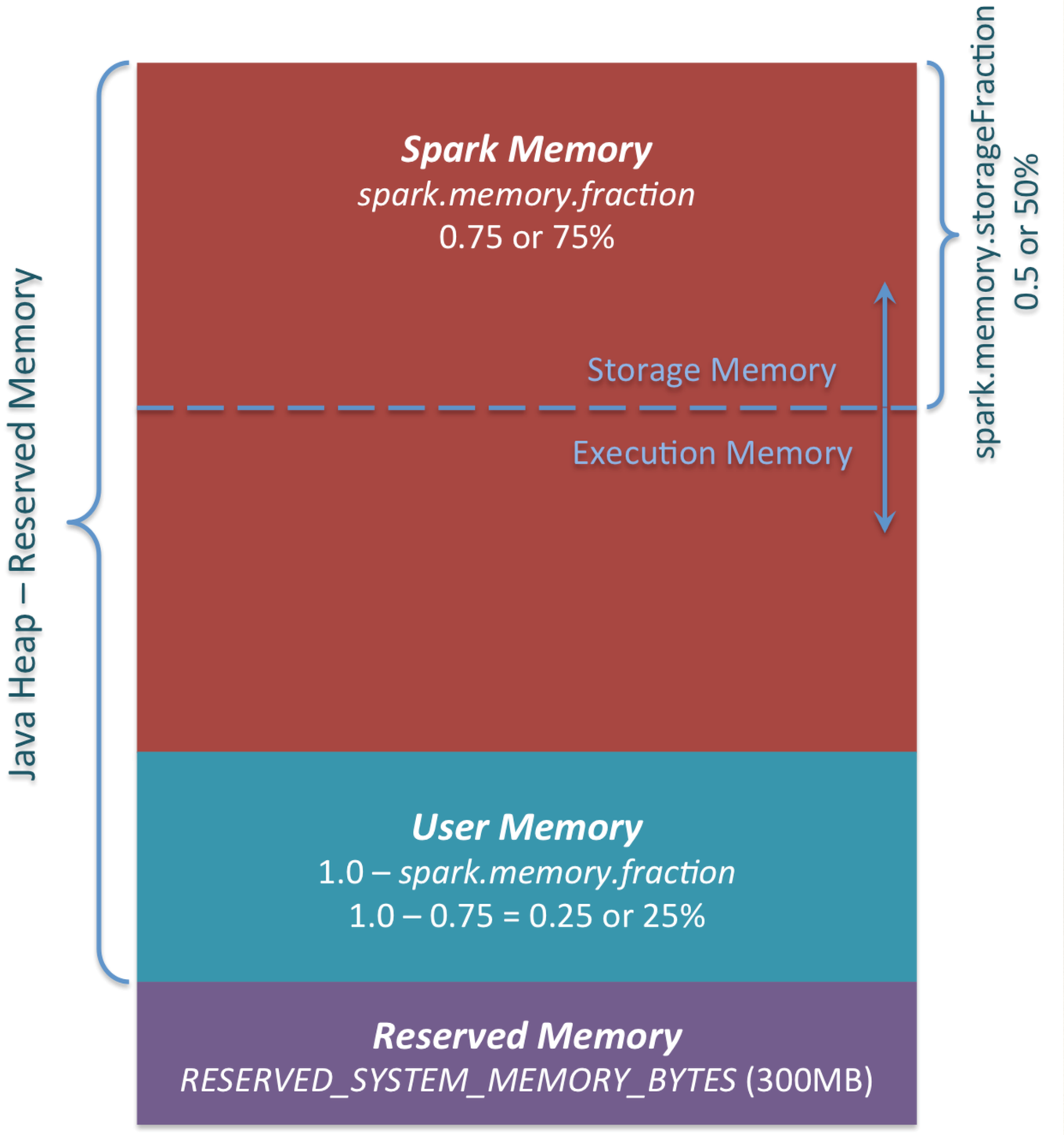


source: apache.org

LRU Cache

- **Least Recently Used Cache**
  - Picks the data that is least recently used and removes it in order to make space for the new data.
  - The priority of the data in the cache changes according to the need of that data

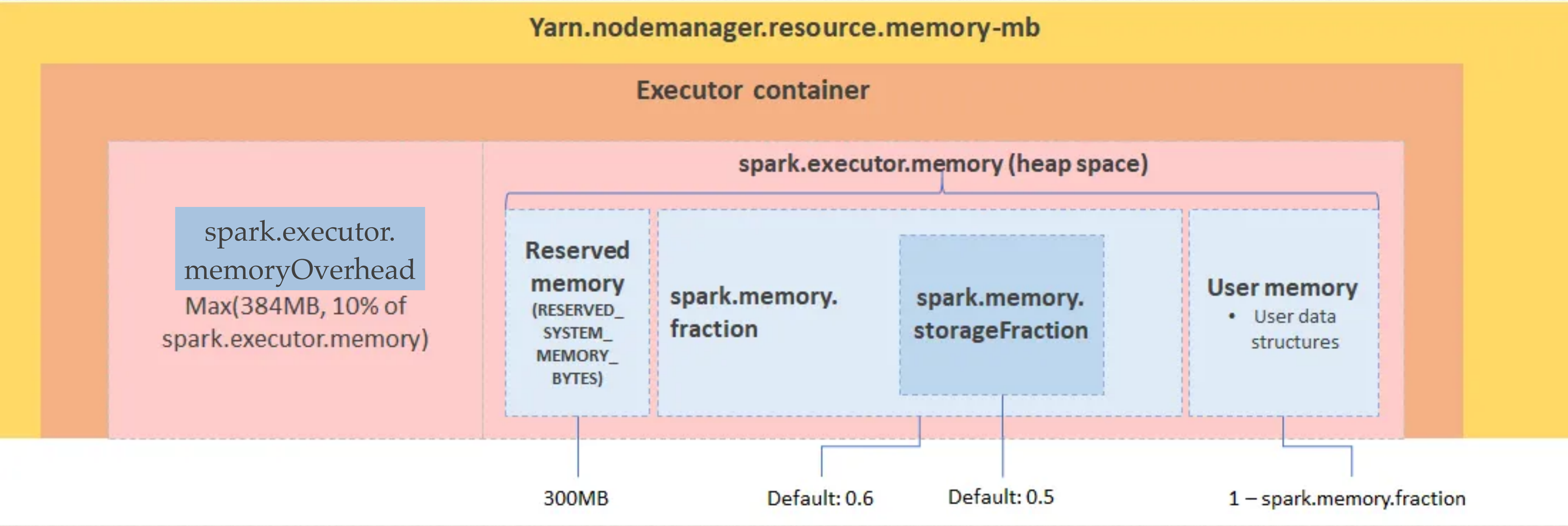
The default value of Spark Memory is 0.6 in Spark v4.2.0



source

On Heap Memory

- **Storage Memory**: mainly used to store Spark cache data, such as RDD cache, Unroll data, and so on.
- **Execution Memory**: mainly used to store temporary data in the calculation process of Shuffle, Join, Sort, Aggregation, etc.
- **User Memory**: mainly used to store the data needed for RDD conversion operations, such as the information for RDD dependency.
- **Reserved Memory**: reserved for the system and is used to store Spark's internal object


Yarn.nodemanager.resource.memory-mb
Executor container
spark.executor.memoryOverhead
Max(384MB, 10% of spark.executor.memory)
spark.executor.memory (heap space)
Reserved memory (RESERVED_SYSTEM_MEMORY_BYTES)
spark.memory.fraction
spark.memory.storageFraction
User memory
• User data structures
300MB
Default: 0.6
Default: 0.5
1 – spark.memory.fraction

source

If your executor memory is 5 GB,

- spark.executor.memory = 5 GB = 5120MB
- default memory overhead = max(10% of spark.executor.memory, 384 MB)
  - max(5 GB * 1024 MB * 0.1, 384 MB) = max(512 MB, 384 MB) = 512 MB.
- YARN/Kubernetes executor container: 5120 + 512 = 5632 MB
- heap space: 5120MB
  - **reserved memory = 300 MB**
  - heap space - reserved memory = 5120 - 300 = 4820 MB
  - **spark.memory.fraction = 0.6 (default)**
    - 4820 MB * 0.6 = 2892 MB
  - **spark.memory.storageFaction = 0.5 (default)**
    - 2892 MB * 0.5 = 1446 MB
  - **user memory** = (1- spark.memory.fraction) * 4820 MB = 1928 MB

Two Axes of Apache Spark Optimization

- Compute efficiency within each task
- **Data movement across the cluster**

Use SparkSQL

- Use SparkSQL to express your queries more effectively if you need to filter, group, or aggregate data. Most of the time, **SparkSQL can reduce shuffling and improve the execution of these queries**.
- SparkSQL provides support for both reading and writing Parquet files that automatically preserves the schema of the original data.

Shuffle in Spark

- A shuffle redistributes records so that data with the same partitioning key reaches the same output partition.
- Operations such as joins, aggregations, distinct, sorting, and repartitioning may require this redistribution.
- A shuffle creates a wide dependency and usually separates the computation into different stages.

Data shuffling is a performance killer

- costly process as it leads to network data transfer by data travel between nodes, High disk I/O and Rearranging files operations.
- The performance of Spark applications can then be enhanced by being aware of these elements and avoiding shuffling whenever possible.

Causes of Shuffle

- Operations that commonly introduce shuffle
  - groupBy, groupByKey, reduceByKey, and aggregations by key
  - Joins when the inputs do not already have compatible partitioning
  - distinct, global sorting, and many window operations
  - repartition() and repartitionByRange()
- Conditions that make shuffle more expensive
  - Data skew
  - Too few or too many shuffle partitions
  - Large records or wide rows
  - Slow disks, limited network bandwidth, or insufficient execution memory

Transformation Optimization (1)

- Use sortWithinPartitions() when only local ordering inside each partition is required. It does not create a globally sorted result.
- Use repartitionByRange() when downstream operations benefit from range-based partitioning. This operation performs a shuffle.
- If ordered records are required within each range partition, follow it with sortWithinPartitions().

Transformation Optimization (2)

- **Filter early**: Reduce the number of rows before expensive joins, windows, and aggregations.
- **Project early**: Keep only the columns required by downstream operations.
- **Use built-in aggregate functions**: They support partial aggregation and are generally preferable to collecting all values for a key.
- **Treat window functions as expensive**: Partitioning and ordering requirements commonly introduce shuffle and sort operations.
- Verify the actual plan with explain() rather than assuming an operation is shuffle-free.

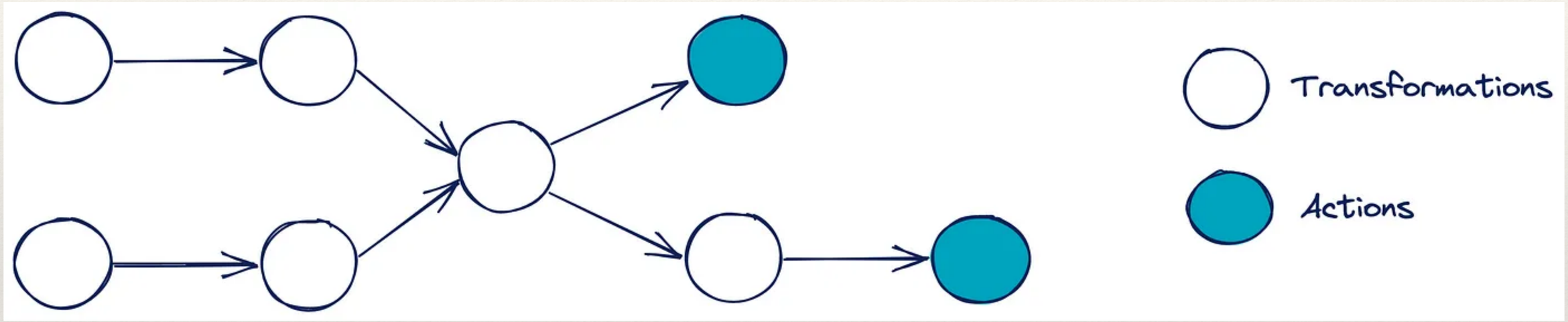
Caching in Spark

```
df.cache()
```

Caching in Spark

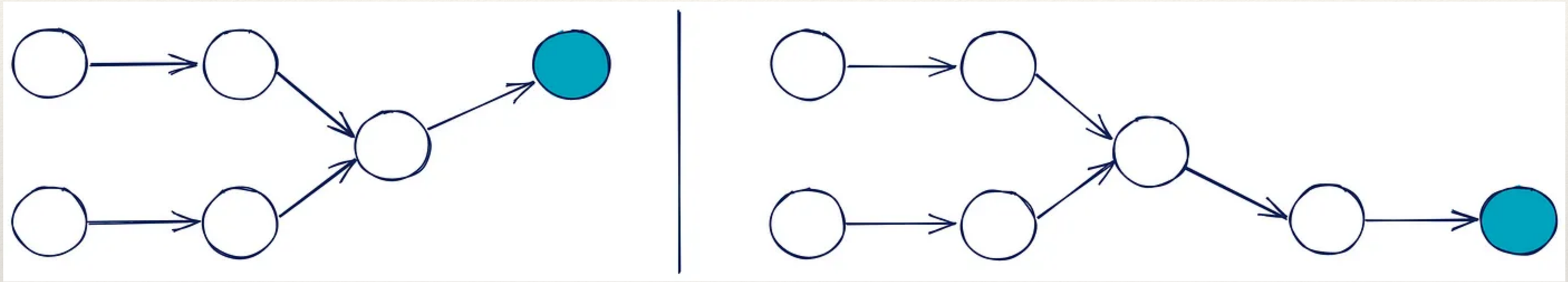
- It is important to keep in mind that caching requires careful planning, because it utilizes the memory resources of Spark's worker nodes, which perform such tasks as executing computations and storing data.
- If the data set is significantly larger than the available memory, or you're caching RDDs or DataFrames without reusing them in subsequent steps, the potential overflow and other memory management issues could introduce bottlenecks in performance.

Caching in Spark



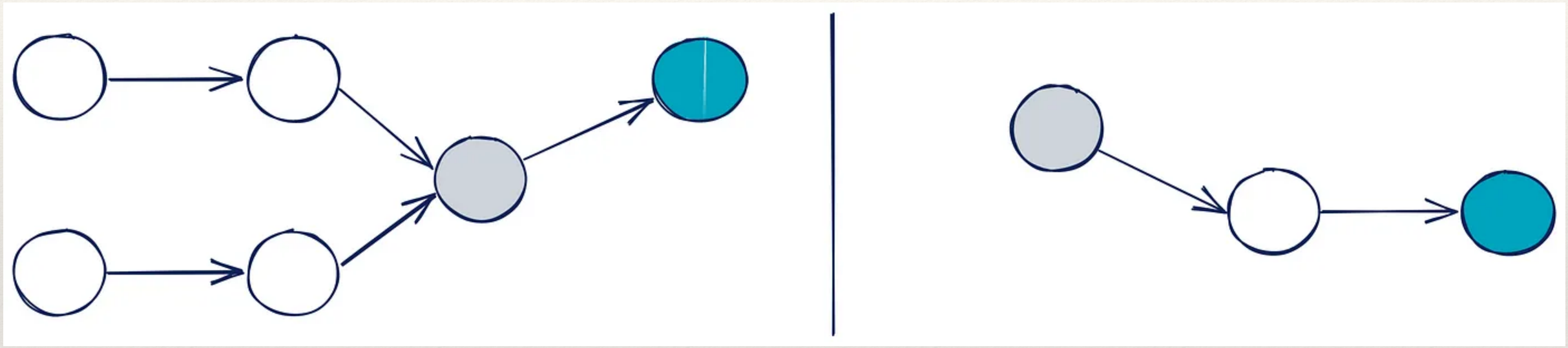
source

Caching in Spark



source

Caching in Spark



source

Persist in Spark

- cache() is shorthand for persistence using the API's default storage level.
- persist(storageLevel) lets you choose a specific storage level.
- For RDDs, the default is MEMORY_ONLY.
- For DataFrames, the default is MEMORY_AND_DISK_DESER.

Persist in Spark

```
val rdd = sc.textFile("hdfs://...")
val processedRdd = rdd.map(...).filter(...)

// Persist the processed RDD in memory
processedRdd.persist()

// Perform multiple actions on the persisted RDD
processedRdd.count()
processedRdd.collect()
```

RDD Persistence

Storage Level	Description	Not available in Python
MEMORY_ONLY	Stores the RDD as deserialized Java objects in the JVM heap. Partitions that do not fit in memory are recomputed when needed.	
MEMORY_ONLY_SER	Stores the RDD as serialized objects in JVM memory. This reduces memory usage but requires deserialization when the data is accessed.	X
MEMORY_AND_DISK	Stores the RDD as deserialized objects in memory and stores partitions that do not fit on disk.	
MEMORY_AND_DISK_SER	Similar to MEMORY_AND_DISK, but stores the RDD as serialized objects in memory.	X
DISK_ONLY	Stores the RDD only on disk and reads it when needed.	
OFF_HEAP	Stores the RDD outside the JVM heap in serialized form. Off-heap memory must be enabled and configured separately.	

* _SER refers to how data is stored in memory; data written to disk is serialized in either case.

When to use Persist

- Use persist when you have a job that involves repeated operations on the same dataset.
- Without persist, each action in Spark r**ecomputes the RDD from scratch**, which can be inefficient if the RDD is large and the computation is complex.
- Persistence avoids repeated lineage computation after the dataset has been materialized by its first action.

Checkpoint in Spark

- The checkpoint method is used to truncate the lineage of an RDD, which means that Spark will **save the RDD to a reliable storage (HDFS, S3, etc.)**, and from that point onwards, Spark will read the RDD from the checkpointed file rather than recomputing it from the original source.
- This is **useful for long-running jobs** where there is a risk of failure and recomputing the RDD from the beginning would be costly.

Checkpoint in Spark

```
val rdd = sc.textFile("hdfs://...")
val processedRdd = rdd.map(...).filter(...)

// Checkpoint the processed RDD
sc.setCheckpointDir("/path/to/checkpoint/dir")
processedRdd.checkpoint()

// Action to trigger the checkpoint
processedRdd.count()
```

When to use Checkpoint

- Use checkpoint when:
  - you have a job where intermediate stages of the computation are costly to recompute
  - and there is a risk of node failures or other issues that could cause the job to restart.
- Checkpointing helps **ensure fault tolerance by breaking the lineage of RDD transformations.**

Cache vs Checkpoint

- Cache materializes the RDD and keeps it in memory (and/or disk). But the lineage (computing chain) of RDD (that is, seq of operations that generated the RDD) will be remembered, so that if there are node failures and parts of the cached RDDs are lost, they can be regenerated.
- However, checkpoint saves the RDD to an HDFS file and **actually forgets the lineage completely**. This allows long lineages to be truncated and the data to be saved reliably in HDFS, which is naturally fault tolerant by replication.

Combining Persist and Checkpoint

```
val rdd = sc.textFile("hdfs://...")
val processedRdd = rdd.map(...).filter(...)

// Persist the processed RDD in memory
processedRdd.persist()

// Checkpoint the processed RDD
sc.setCheckpointDir("/path/to/checkpoint/dir")
processedRdd.checkpoint()

// Action to trigger the persist and checkpoint
processedRdd.count()
```

Data Partitioning in Spark

- Spark determines the initial number of partitions based on the data source and operation. It also provides options for repartitioning data by a selected key or partition count.
- One of the most important factors affecting the efficiency of parallel processing is the number of partitions.
  - If there aren't enough partitions, the available memory and resources may be underutilized.
  - On the other hand, too many partitions can lead to increased performance overhead due to task scheduling and coordination.
  - The optimal number of partitions depends on **the data size, operation, and available cluster resources.**

repartition() and coalesce()

```
df = df.repartition(200)     # repartition method

df = df.coalesce(200)        # coalesce method
```

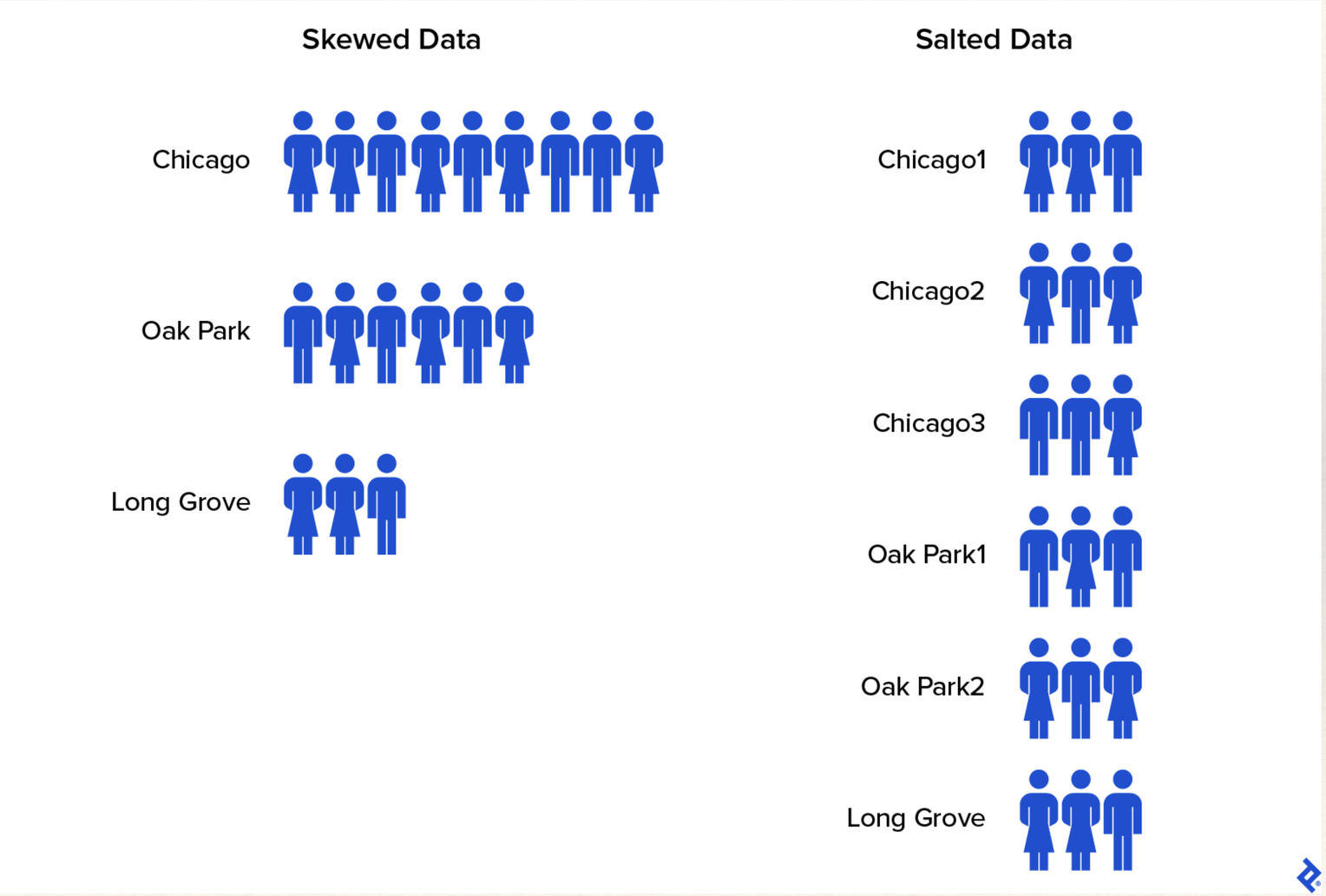
repartition() and coalesce()

- The repartition() method increases or decreases the number of partitions in an RDD or DataFrame and **performs a full shuffle of the data across the cluster**, which can be costly in terms of processing and network latency.
- The coalesce() method decreases the number of partitions in an RDD or DataFrame and, unlike repartition(), **does not perform a full shuffle**, instead combining adjacent partitions to reduce the overall number.

Handling Data Skews

- **Partitioning**
  - Properly partitioning data can help distribute the workload evenly across nodes.
- **Sampling**
  - Using sampling techniques to identify skewed keys and apply custom partitioning or filtering strategies.
- **Aggregation**
  - Using alternative aggregation strategies, such as pre-aggregation or partial aggregation, to reduce the impact of data skew.

Salting

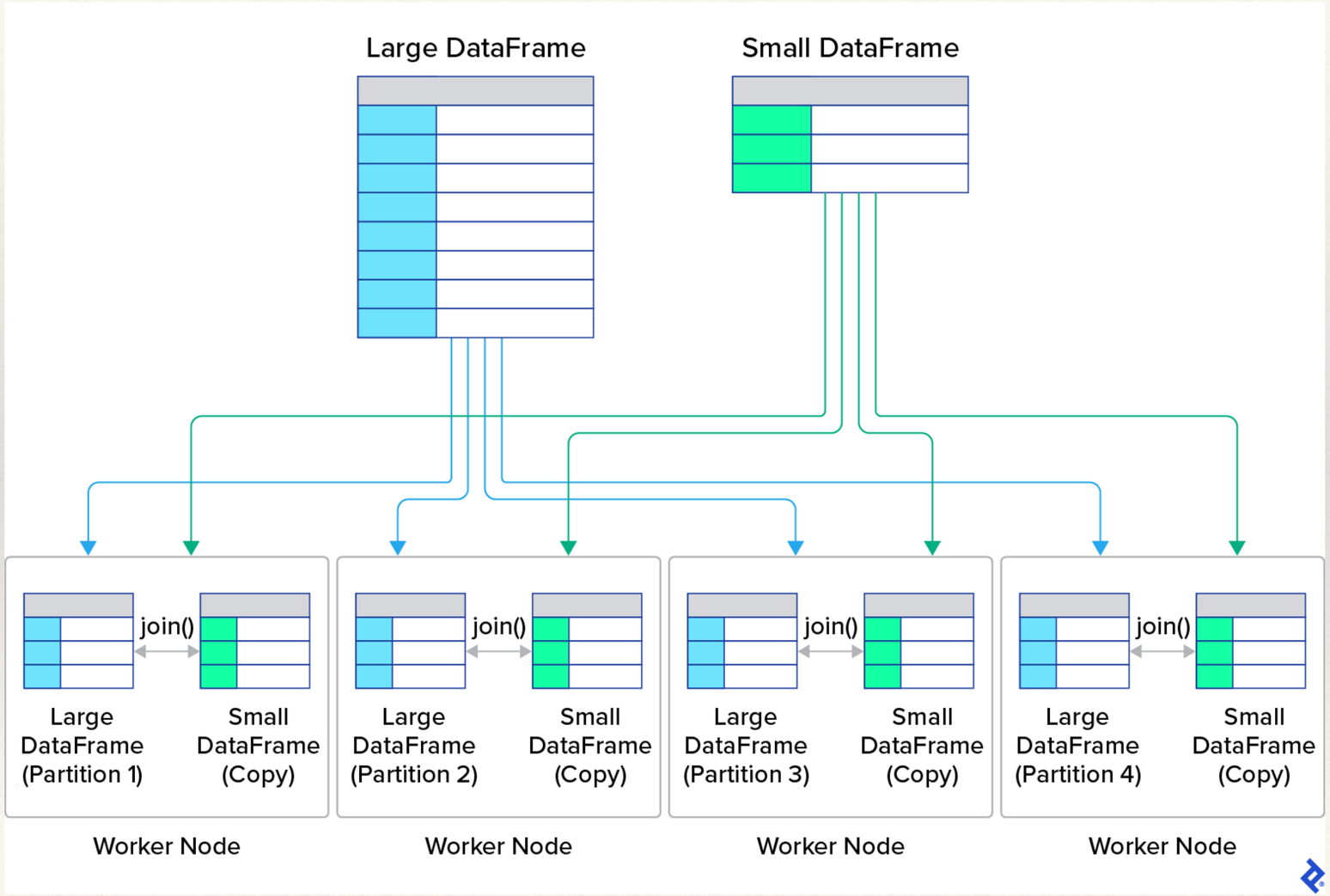


source

Broadcasting

- A join() is a common operation in which two data sets are combined based on one or more common keys. Rows from two different data sets can be merged into a single data set by matching values in the specified columns. Because data shuffling across multiple nodes is required, a join() can be a costly operation in terms of network latency.
- In scenarios in which **a small data set is being joined with a larger data set**, Spark offers an optimization technique called broadcasting.
- **If one of the data sets is small enough to fit into the memory of each worker node, it can be sent to all nodes, reducing the need for costly shuffle operations.** The join() operation simply happens locally on each node.

Broadcasting



source

Broadcasting

```
from pyspark.sql.functions import broadcast
df1.join(broadcast(df2), 'id')
# must be small enough to fit into the memory of each worker node;
a DataFrame that is too large will cause out-of-memory errors.
```

Filtering Unused Data

- When working with high-dimensional data, minimizing computational overhead is essential. Any rows or columns that are not absolutely required should be removed. Two key techniques that reduce computational complexity and memory usage are early filtering and column pruning:
  - **Early filtering**: Filtering operations should be applied as early as possible in the data processing pipeline. This **cuts down on the number of rows** that need to be processed in subsequent transformations, reducing the overall computational load and memory resources.
  - **Column pruning**: Many computations involve only a subset of columns in a data set. **Columns that are not necessary for data processing should be removed.** Column pruning can significantly decrease the amount of data that needs to be processed and stored.

Filtering

```
df = df.select(“name”, ”age”).filter(df[”age”] > 21)
```

```
df = df.filter(df[”age”] > 21).select(”name”, ”age”)
```

Spark Web UI

Spark Web UI

- The driver exposes the live Spark Web UI, normally beginning at port 4040.
- On YARN, the Resource Manager or Application Master proxy can provide a link to the application UI. Other cluster managers expose it differently.
- The UI displays application-level jobs, stages, tasks, executors, storage, SQL plans, and runtime metrics.

Spark History Server

- The Spark History Server reconstructs an application UI from persisted event logs.
- It can list completed and incomplete applications when their event logs are available.
- Some live-only links or external logs may no longer be accessible, but the core job, stage, task, executor, storage, and SQL events can be replayed from the log.

Spark History Server

- **./sbin/start-history-server.sh**

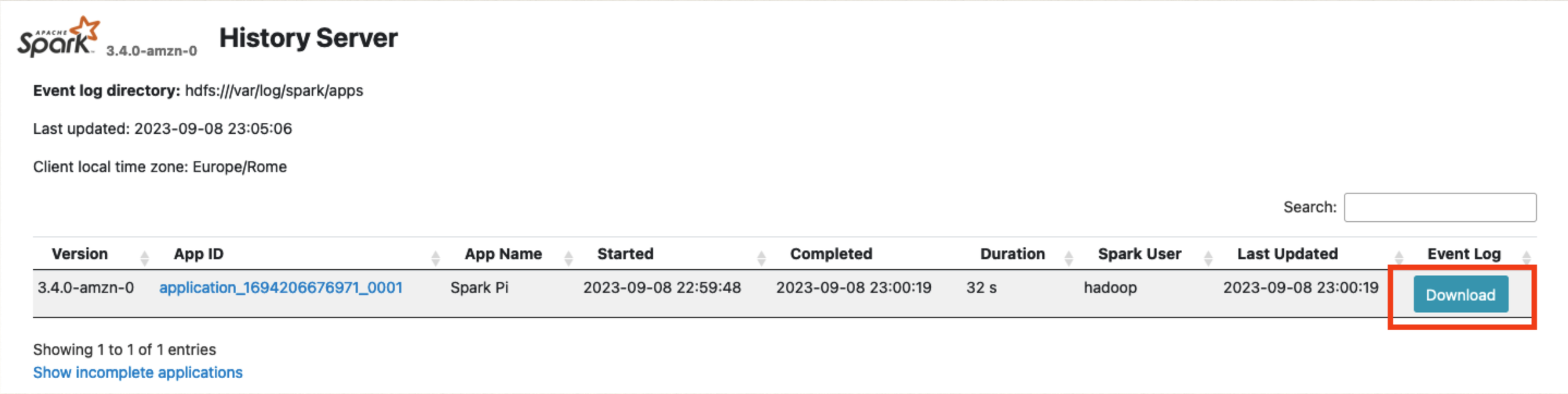
- **History Server**
  - spark.history.fs.logDirectory hdfs://namenode/shared/spark-logs

- **Spark applications**
  - spark.eventLog.enabled true
  - spark.eventLog.dir hdfs://namenode/shared/spark-logs

- The History Server and Spark applications must use the same shared event-log location.

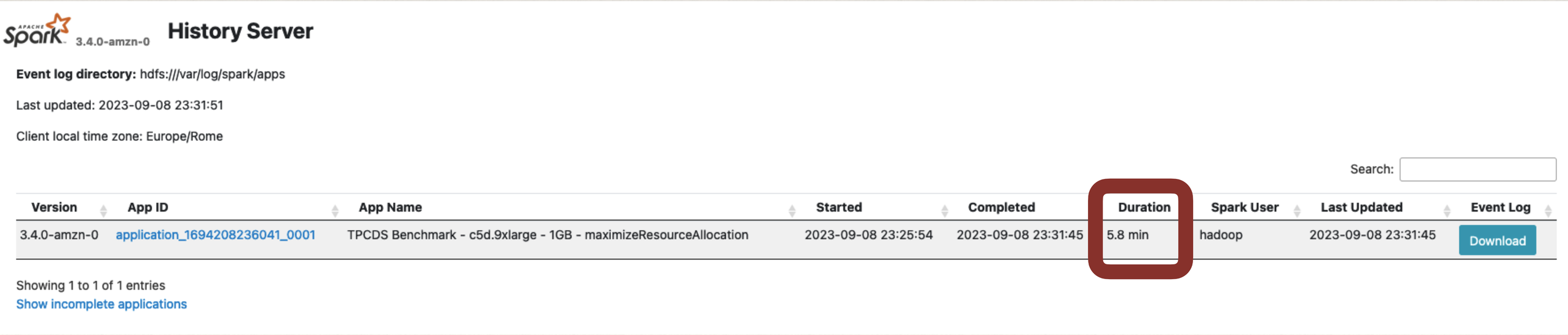
Retrieve Spark Event Logs

- On Amazon EMR, first verify the effective spark.eventLog.dir in the Environment tab or Spark configuration.
- The event-log location may differ by EMR release and deployment type. For access after cluster termination, configure persistent UI or log archiving to Amazon S3.



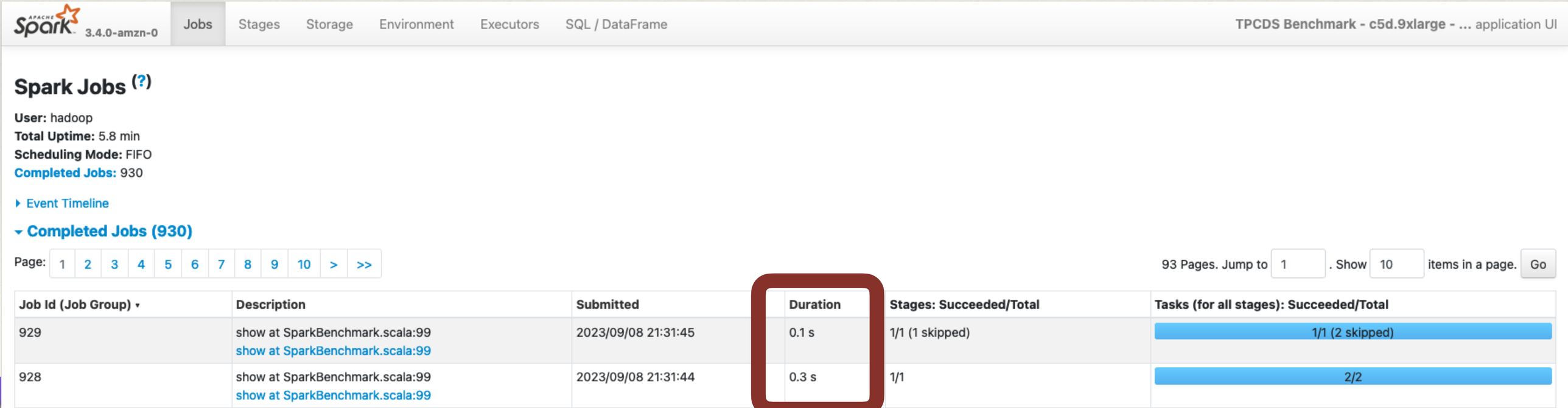
Applications in Spark Web UII

- Application Duration measures the elapsed time from application start to completion.
- Use it for coarse comparisons only. Meaningful benchmark comparisons require equivalent input data, code, Spark configuration, and cluster resources.



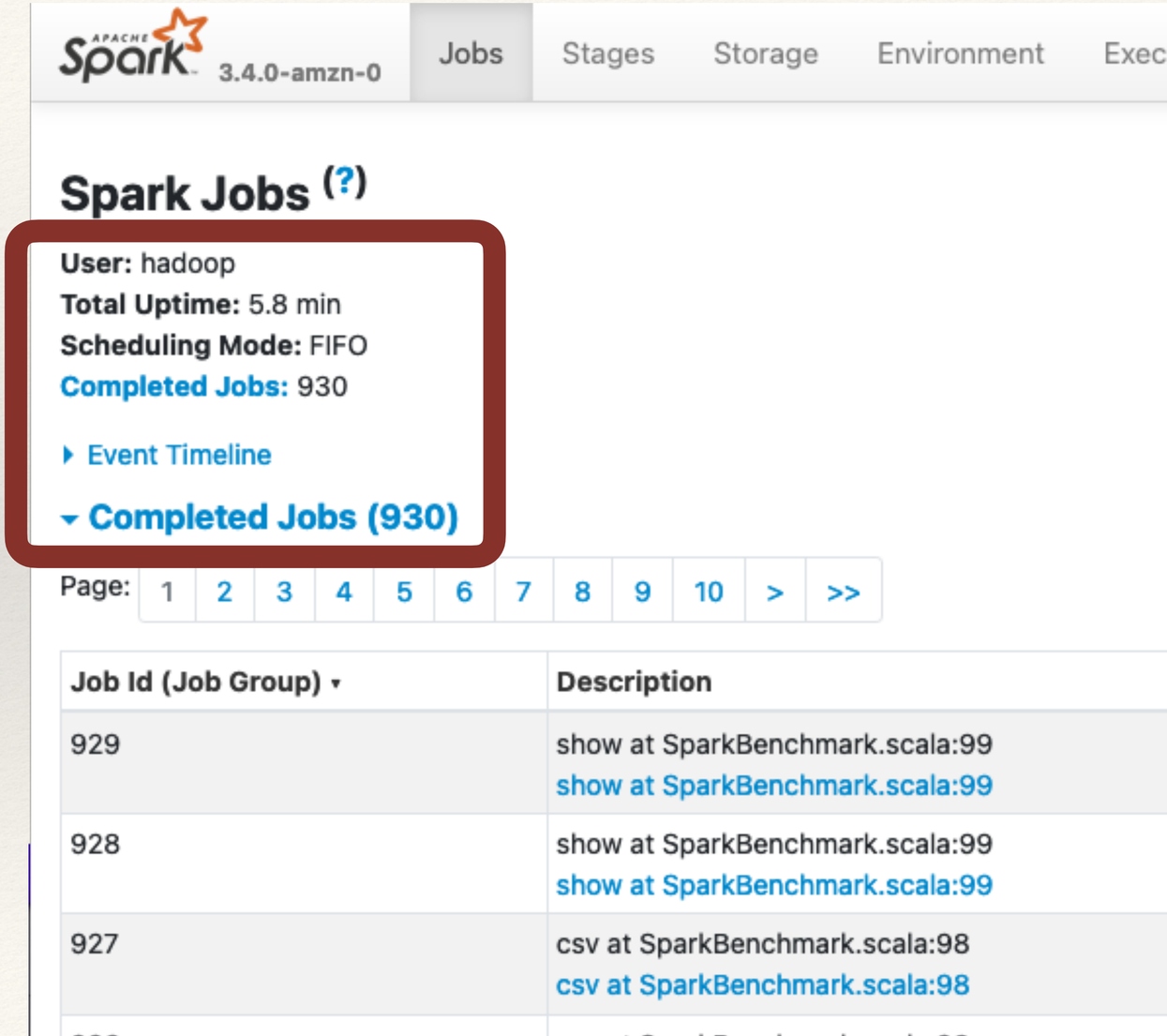
Jobs in Spark Web UI (1)

- Sort jobs by Duration to identify expensive jobs.
- Open an individual job to inspect its stages and DAG.
- Compare jobs across runs only when the input, code, configuration, and cluster resources are controlled.



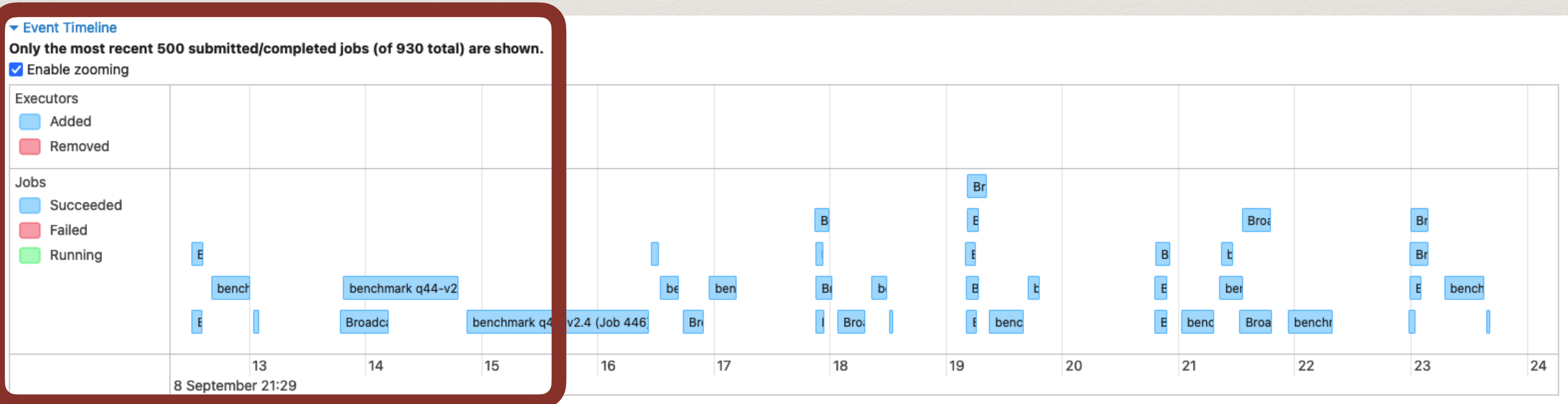
Jobs in Spark Web UI (2)

- **User**: The account that launched the application.
- **Total Uptime**: Elapsed time since the Spark application started.
- **Scheduling Mode**: The scheduler used by the application. Spark uses FIFO by default and can be configured to use FAIR scheduling.



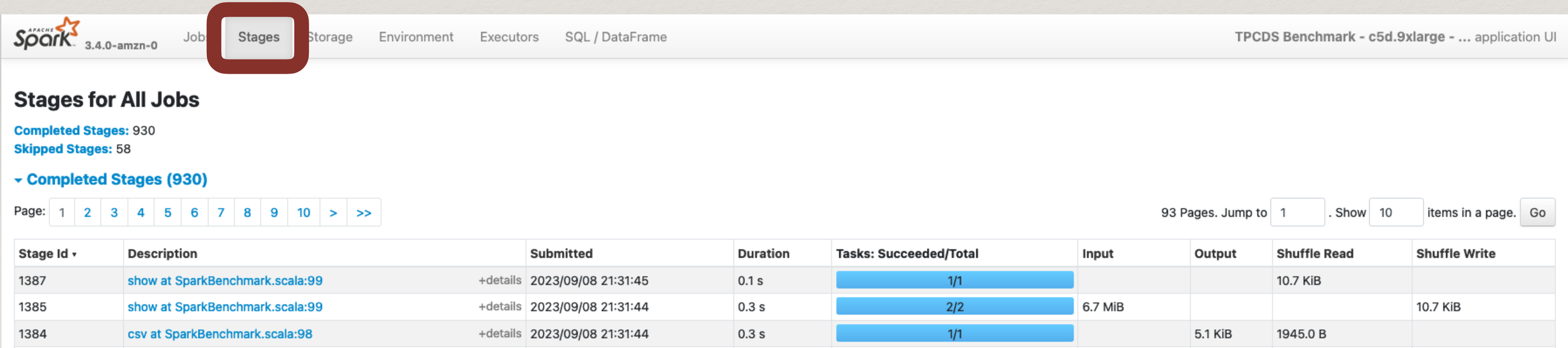
Jobs in Spark Web UI (3)

- Expand the Event Timeline to review:
  - when jobs started and completed
  - when executors were added or removed
  - idle gaps between jobs
  - delays caused by dynamic allocation or cluster scaling



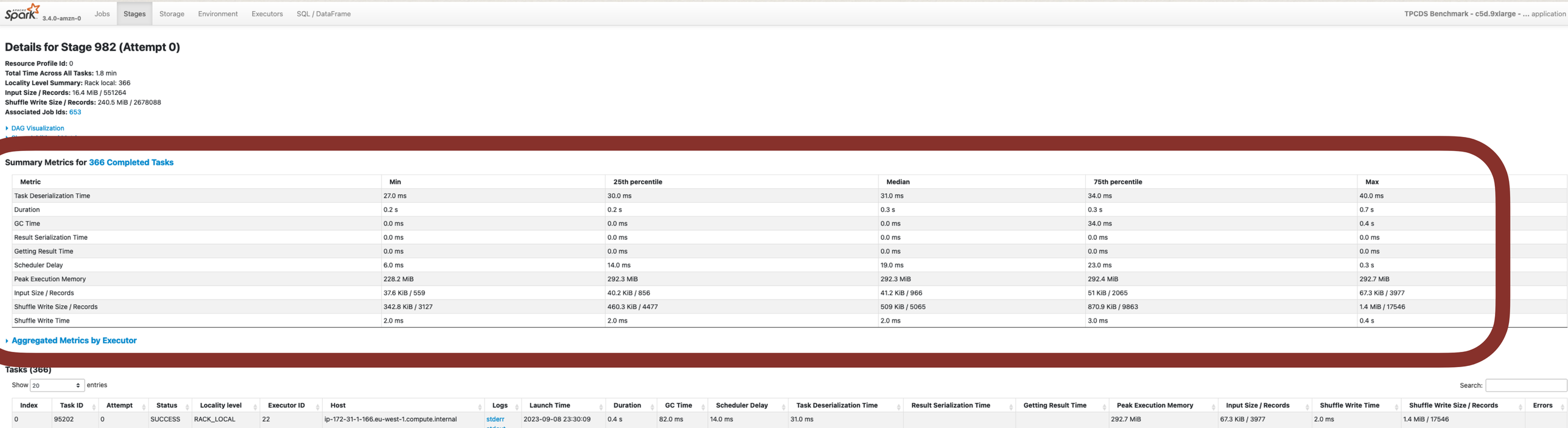
Stages in Spark Web UI (1)

- The Stages tab lists every stage in the application.
- You can also open a job from the Jobs tab to view only the stages that belong to that job.



Stages in Spark Web UI (2)

- Open a stage to inspect task-level distributions for:
  - Duration and scheduler delay
  - Input and output size
  - Shuffle read and write
  - GC time
  - Memory and disk spill
  - Peak execution memory



Spark 3.4.0-amzn-0 Jobs Stages Storage Environment Executors SQL / DataFrame TPCDS Benchmark - c5d.9xlarge - ... application

Details for Stage 982 (Attempt 0)

Resource Profile Id: 0
Total Time Across All Tasks: 1.8 min
Locality Level Summary: Rack local: 366
Input Size / Records: 16.4 MiB / 551264
Shuffle Write Size / Records: 240.5 MiB / 2678088
Associated Job Ids: 653

DAG Visualization

Summary Metrics for 366 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Task Deserialization Time	27.0 ms	30.0 ms	31.0 ms	34.0 ms	40.0 ms
Duration	0.2 s	0.2 s	0.3 s	0.3 s	0.7 s
GC Time	0.0 ms	0.0 ms	0.0 ms	34.0 ms	0.4 s
Result Serialization Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	0.0 ms
Getting Result Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	0.0 ms
Scheduler Delay	6.0 ms	14.0 ms	19.0 ms	23.0 ms	0.3 s
Peak Execution Memory	228.2 MiB	292.3 MiB	292.3 MiB	292.4 MiB	292.7 MiB
Input Size / Records	37.6 KiB / 559	40.2 KiB / 856	41.2 KiB / 966	51 KiB / 2065	67.3 KiB / 3977
Shuffle Write Size / Records	342.8 KiB / 3127	460.3 KiB / 4477	509 KiB / 5065	870.9 KiB / 9863	1.4 MiB / 17546
Shuffle Write Time	2.0 ms	2.0 ms	2.0 ms	3.0 ms	0.4 s

Aggregated Metrics by Executor

Tasks (366)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Scheduler Delay	Task Deserialization Time	Result Serialization Time	Getting Result Time	Peak Execution Memory	Input Size / Records	Shuffle Write Time	Shuffle Write Size / Records	Errors
0	95202	0	SUCCESS	RACK_LOCAL	22	ip-172-31-1-166.eu-west-1.compute.internal	stderr	2023-09-08 23:30:09	0.4 s	82.0 ms	14.0 ms	31.0 ms			292.7 MiB	67.3 KiB / 3977	2.0 ms	1.4 MiB / 17546	

Details for Stage 982 (Attempt 0)

Resource Profile Id: 0
Total Time Across All Tasks: 1.8 min
Locality Level Summary: Rack local: 366
Input Size / Records: 16.4 MiB / 551264
Shuffle Write Size / Records: 240.5 MiB / 2678088
Associated Job Ids: 653

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 366 Completed Tasks

Metric	Min	25th percentile
Task Deserialization Time	27.0 ms	30.0 ms
Duration	0.2 s	0.2 s
GC Time	0.0 ms	0.0 ms
Result Serialization Time	0.0 ms	0.0 ms
Getting Result Time	0.0 ms	0.0 ms
Scheduler Delay	6.0 ms	14.0 ms
Peak Execution Memory	228.2 MiB	292.3 MiB
Input Size / Records	37.6 KiB / 559	40.2 KiB / 856
Shuffle Write Size / Records	342.8 KiB / 3127	460.3 KiB / 4477
Shuffle Write Time	2.0 ms	2.0 ms

▸ Aggregated Metrics by Executor

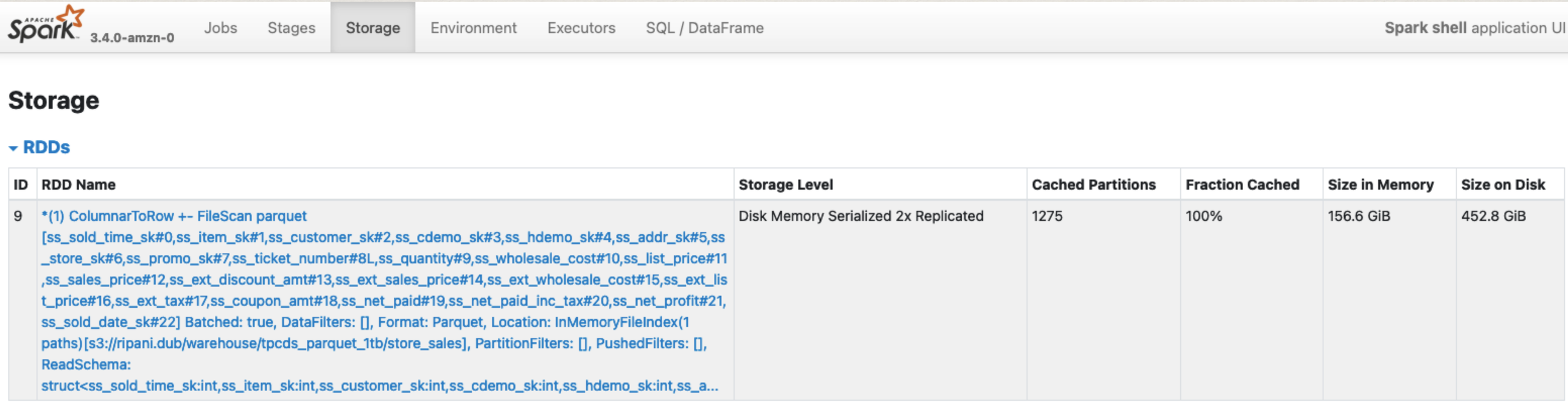
Tasks (366)

Show 20 entries

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Tim
0	95202	0	SUCCESS	RACK_LOCAL	22	ip-172-31-1-166.eu-west-1.compute.internal	stderr stdout	2023-09-08 23:30:09	0.4 s	82.0 ms

Storage in Spark Web UI

- The Storage page** contains **information about RDD blocks that have been cached or persisted in memory or on the local disks of the cluster**. This page will show some details only if you explicitly invoke a persist or cache operation against a Spark Dataframe. More in detail, the page shows for each RDD:



RDD Storage Info for *(1) ColumnarToRow +- FileScan parquet [ss_sold_time_sk#0,ss_item_sk#1,ss_customer_sk#2,ss_cdemo_sk#...

Storage Level: Disk Memory Serialized 2x Replicated
Cached Partitions: 1278
Total Partitions: 1278
Memory Size: 156.6 GiB
Disk Size: 459.9 GiB

Data Distribution on 6 Executors

Host	On Heap Memory Usage	Off Heap Memory Usage	Disk Usage
ip-172-31-3-222.eu-west-1.compute.internal:44907	26.2 GiB (336.4 MiB Remaining)	0.0 B (0.0 B Remaining)	89.1 GiB
ip-172-31-3-244.eu-west-1.compute.internal:43799	26.3 GiB (267.1 MiB Remaining)	0.0 B (0.0 B Remaining)	44.3 GiB
ip-172-31-3-170.eu-west-1.compute.internal:46697	25.8 GiB (751.3 MiB Remaining)	0.0 B (0.0 B Remaining)	91.4 GiB
ip-172-31-3-36.eu-west-1.compute.internal:46023	26.1 GiB (476.3 MiB Remaining)	0.0 B (0.0 B Remaining)	94.3 GiB
ip-172-31-3-39.eu-west-1.compute.internal:43111	26.0 GiB (594.8 MiB Remaining)	0.0 B (0.0 B Remaining)	49.5 GiB
ip-172-31-3-196.eu-west-1.compute.internal:39167	26.2 GiB (344.6 MiB Remaining)	0.0 B (0.0 B Remaining)	91.4 GiB

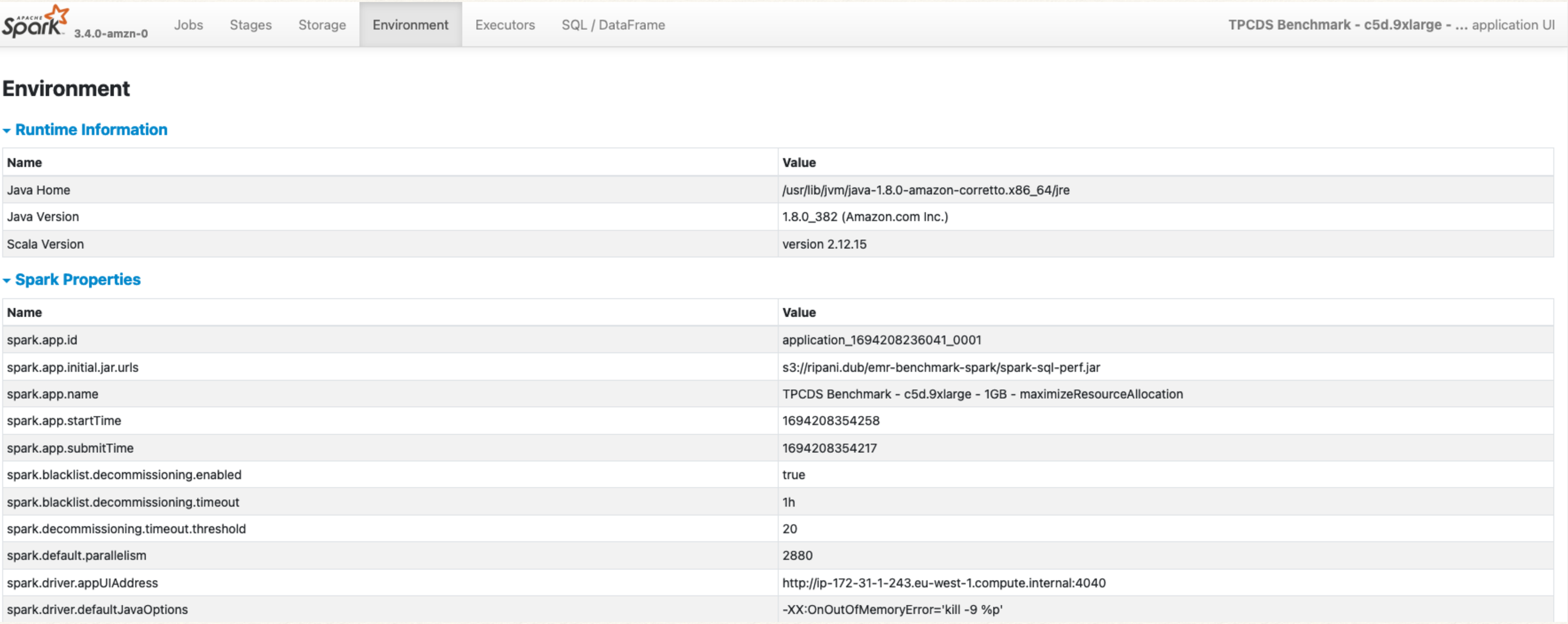
1278 Partitions

Page: < 1 2 3 4 5 6 7 8 9 10 > >>

128 Pages. Jump to 5 . Show 10 items in a page. Go

Block Name ▾	Storage Level	Size in Memory	Size on Disk	Executors
rdd_9_962	Disk Serialized 2x Replicated	0.0 B	388.9 MiB	ip-172-31-3-170.eu-west-1.compute.internal:46697 ip-172-31-3-39.eu-west-1.compute.internal:43111
rdd_9_961	Disk Serialized 2x Replicated	0.0 B	389.1 MiB	ip-172-31-3-196.eu-west-1.compute.internal:39167 ip-172-31-3-36.eu-west-1.compute.internal:46023
rdd_9_960	Disk Serialized 2x Replicated	0.0 B	389.1 MiB	ip-172-31-3-170.eu-west-1.compute.internal:46697 ip-172-31-3-39.eu-west-1.compute.internal:43111
rdd_9_96	Memory Serialized 2x Replicated	613.0 MiB	0.0 B	ip-172-31-3-170.eu-west-1.compute.internal:46697 ip-172-31-3-36.eu-west-1.compute.internal:46023
rdd_9_959	Disk Serialized 2x Replicated	0.0 B	389.5 MiB	ip-172-31-3-170.eu-west-1.compute.internal:46697 ip-172-31-3-36.eu-west-1.compute.internal:46023
rdd_9_958	Disk Serialized 2x Replicated	0.0 B	389.7 MiB	ip-172-31-3-222.eu-west-1.compute.internal:44907 ip-172-31-3-39.eu-west-1.compute.internal:43111
rdd_9_957	Disk Serialized 2x Replicated	0.0 B	388.8 MiB	ip-172-31-3-170.eu-west-1.compute.internal:46697 ip-172-31-3-222.eu-west-1.compute.internal:44907
rdd_9_956	Disk Serialized 2x Replicated	0.0 B	389.9 MiB	ip-172-31-3-222.eu-west-1.compute.internal:44907 ip-172-31-3-39.eu-west-1.compute.internal:43111

Environment in Spark Web UI



Executors in Spark Web UI

Executors

Show Additional Metrics

Summary

	RDD Blocks	Storage Memory	On Heap Storage Memory	Off Heap Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(7)	2556	156.6 GiB / 169.8 GiB	156.6 GiB / 169.8 GiB	0.0 B / 0.0 B	459.9 GiB	96	0	0	3104	3104	11.4 h (46 min)	127.6 GiB	62.4 KiB	62.4 KiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B / 0.0 B	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(7)	2556	156.6 GiB / 169.8 GiB	156.6 GiB / 169.8 GiB	0.0 B / 0.0 B	459.9 GiB	96	0	0	3104	3104	11.4 h (46 min)	127.6 GiB	62.4 KiB	62.4 KiB	0

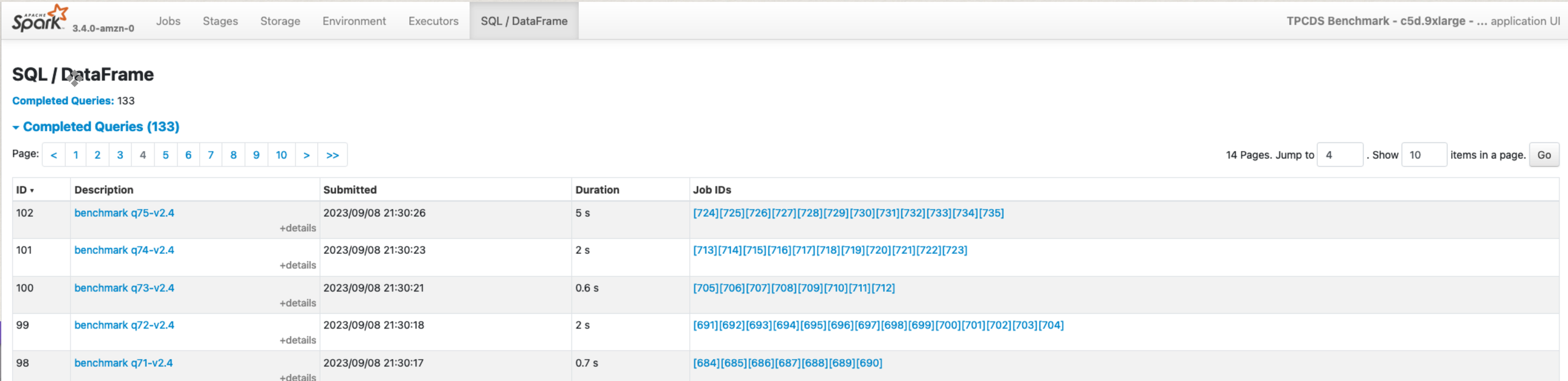
Executors

Show 20 entries Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	On Heap Storage Memory	Off Heap Storage Memory	Peak JVM Memory OnHeap / OffHeap	Peak Execution Memory OnHeap / OffHeap	Peak Storage Memory OnHeap / OffHeap	Peak Pool Memory Direct / Mapped	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump
driver	ip-172-31-3-68.eu-west-1.compute.internal:41281	Active	0	66.9 KiB / 10.5 GiB	66.9 KiB / 10.5 GiB	0.0 B / 0.0 B	920.2 MiB / 274.8 MiB	0.0 B / 0.0 B	1012.2 KiB / 0.0 B	6.5 MiB / 0.0 B	0.0 B	0	0	0	0	0	18 min (0.8 s)	0.0 B	0.0 B	0.0 B		Thread Dump
1	ip-172-31-3-170.eu-west-1.compute.internal:46697	Active	473	25.8 GiB / 26.6 GiB	25.8 GiB / 26.6 GiB	0.0 B / 0.0 B	43.8 GiB / 142.3 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	2.6 GiB / 1.7 GiB	91.4 GiB	16	0	0	679	679	2.1 h (7.6 min)	22.5 GiB	0.0 B	10.6 KiB	stdout stderr	Thread Dump
2	ip-172-31-3-222.eu-west-1.compute.internal:44907	Active	466	26.2 GiB / 26.6 GiB	26.2 GiB / 26.6 GiB	0.0 B / 0.0 B	40.6 GiB / 141.5 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	2.3 GiB / 1.8 GiB	89.1 GiB	16	0	0	701	701	2.1 h (7.3 min)	23.9 GiB	0.0 B	11.4 KiB	stdout stderr	Thread Dump
3	ip-172-31-3-196.eu-west-1.compute.internal:39167	Active	473	26.2 GiB / 26.6 GiB	26.2 GiB / 26.6 GiB	0.0 B / 0.0 B	41.6 GiB / 141.5 MiB	0.0 B / 0.0 B	26.5 GiB / 0.0 B	2.2 GiB / 1.1 GiB	91.4 GiB	16	0	0	680	680	2.1 h (8.0 min)	22.7 GiB	0.0 B	10.9 KiB	stdout stderr	Thread Dump
4	ip-172-31-3-36.eu-west-1.compute.internal:46023	Active	482	26.1 GiB / 26.6 GiB	26.1 GiB / 26.6 GiB	0.0 B / 0.0 B	42.6 GiB / 144.3 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	2.2 GiB / 1.7 GiB	94.3 GiB	16	0	0	674	674	2.1 h (9.2 min)	23.9 GiB	62.4 KiB	11.4 KiB	stdout stderr	Thread Dump
5	ip-172-31-3-244.eu-west-1.compute.internal:43799	Active	317	26.3 GiB / 26.6 GiB	26.3 GiB / 26.6 GiB	0.0 B / 0.0 B	41.8 GiB / 131.7 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	1.7 GiB / 754 MiB	44.3 GiB	16	0	0	178	178	1.4 h (7.7 min)	16.8 GiB	0.0 B	8.7 KiB	stdout stderr	Thread Dump
6	ip-172-31-3-39.eu-west-1.compute.internal:43111	Active	345	26 GiB / 26.6 GiB	26 GiB / 26.6 GiB	0.0 B / 0.0 B	43.6 GiB / 130.3 MiB	0.0 B / 0.0 B	26.5 GiB / 0.0 B	1.9 GiB / 1.1 GiB	49.5 GiB	16	0	0	192	192	1.4 h (6.2 min)	17.8 GiB	0.0 B	9.4 KiB	stdout stderr	Thread Dump

SQL/DataFrame in Spark Web UI

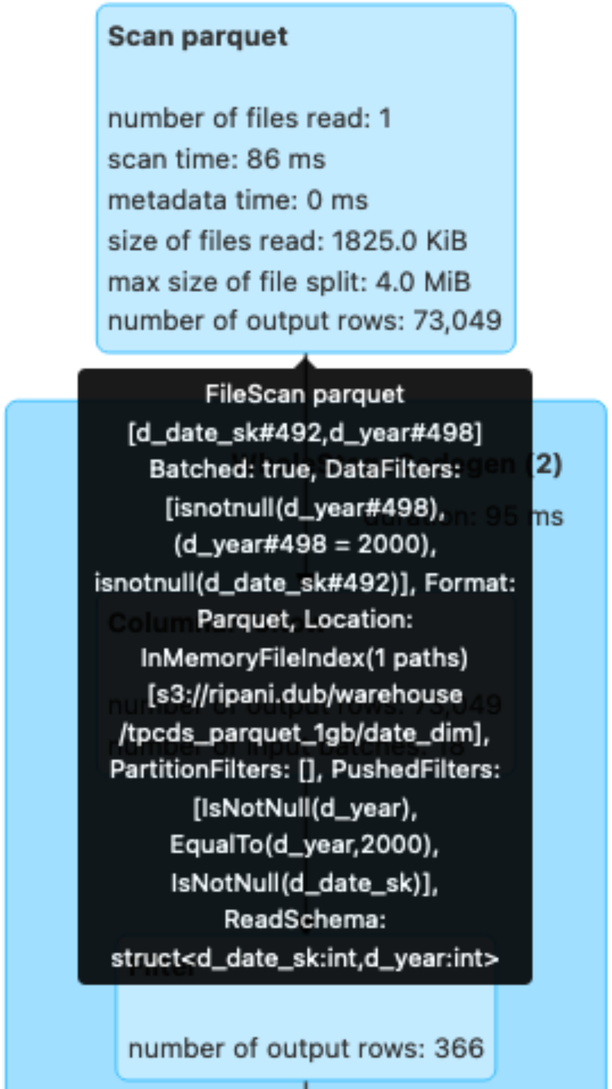
- SQL/DataFrame page summarizes **all Spark Queries executed in a Spark Application.**
- This page is only visible in the UI if your application is using Dataset or DataFrame Spark APIs. (Not when using RDD APIs)



Details for Query 30

Submitted Time: 2023/09/08 21:27:31
Duration: 1 s
Succeeded Jobs: 90 91 92 93 94 95

☐ Show the Stage ID and Task ID that corresponds to the max metric



Details

```
== Parsed Logical Plan ==
+- 'LocalLimit 100
   +- 'Sort ['i_item_id ASC NULLS FIRST], true
      +- 'Aggregate ['i_item_id], ['i_item_id, 'avg('ss_quantity) AS agg1#27406, 'avg('ss_list_price) AS agg2#27407, 'avg('s
         +- 'Filter (((((('ss_sold_date_sk = 'd_date_sk) AND ('ss_item_sk = 'i_item_sk)) AND ('ss_cdemo_sk = 'cd_demo_sk)) AN
            +- 'Join Inner
               :- 'Join Inner
               :  :- 'Join Inner
               :  :  :- 'Join Inner
               :  :  :  :- 'UnresolvedRelation [store_sales], [], false
               :  :  :  +- 'UnresolvedRelation [customer_demographics], [], false
               :  :  +- 'UnresolvedRelation [date_dim], [], false
               :  +- 'UnresolvedRelation [item], [], false
               +- 'UnresolvedRelation [promotion], [], false

== Analyzed Logical Plan ==
i_item_id: string, agg1: double, agg2: decimal(11,6), agg3: decimal(11,6), agg4: decimal(11,6)
GlobalLimit 100
+- LocalLimit 100
   +- Sort [i_item_id#565 ASC NULLS FIRST], true
      +- Aggregate [i_item_id#565], [i_item_id#565, avg(ss_quantity#139) AS agg1#27406, avg(ss_list_price#141) AS agg2#27407
         +- Filter (((((ss_sold_date_sk#152 = d_date_sk#492) AND (ss_item_sk#131 = i_item_sk#564)) AND (ss_cdemo_sk#133 = cd
            +- Join Inner
               :- Join Inner
               :  :- Join Inner
               :  :  :- Join Inner
```

```
:  :  :- Join Inner
:  :  :  :- SubqueryAlias store_sales
:  :  :  :  +- View (`store_sales`,
[ss_sold_time_sk#130,ss_item_sk#131,ss_customer_sk#132,ss_cdemo_sk#133,ss_hdemo_sk#134,ss_addr_sk#135,ss_store_sk#136,ss_pro
:  :  :  :     +- Relation
[ss_sold_time_sk#130,ss_item_sk#131,ss_customer_sk#132,ss_cdemo_sk#133,ss_hdemo_sk#134,ss_addr_sk#135,ss_store_sk#136,ss_pro
:  :  :  +- SubqueryAlias customer_demographics
:  :  :     +- View (`customer_demographics`, [cd_demo_sk#474,cd_gender#475,cd_marital_status#476,cd_educatio
:  :  :        +- Relation [cd_demo_sk#474,cd_gender#475,cd_marital_status#476,cd_education_status#477,cd_pur
:  :  +- SubqueryAlias date_dim
:  :     +- View (`date_dim`,
[d_date_sk#492,d_date_id#493,d_date#494,d_month_seq#495,d_week_seq#496,d_quarter_seq#497,d_year#498,d_dow#499,d_moy#500,d_do
:  :        +- Relation [d_date_sk#492,d_date_id#493,d_date#494,d_month_seq#495,d_week_seq#496,d_quarter_seq#
:  +- SubqueryAlias item
:     +- View (`item`, [i_item_sk#564,i_item_id#565,i_rec_start_date#566,i_rec_end_date#567,i_item_desc#568,i
:        +- Relation [i_item_sk#564,i_item_id#565,i_rec_start_date#566,i_rec_end_date#567,i_item_desc#568,i_cu
+- SubqueryAlias promotion
   +- View (`promotion`, [p_promo_sk#608,p_promo_id#609,p_start_date_sk#610,p_end_date_sk#611,p_item_sk#612,p_
      +- Relation [p_promo_sk#608,p_promo_id#609,p_start_date_sk#610,p_end_date_sk#611,p_item_sk#612,p_cost#61

== Optimized Logical Plan ==
GlobalLimit 100
+- LocalLimit 100
   +- Sort [i_item_id#565 ASC NULLS FIRST], true
      +- Aggregate [i_item_id#565], [i_item_id#565, avg(ss_quantity#139) AS agg1#27406, cast((avg(UnscaledValue(ss_list_price
         +- Project [ss_quantity#139, ss_list_price#141, ss_sales_price#142, ss_coupon_amt#148, i_item_id#565]
            +- Join Inner, (ss_promo_sk#137 = p_promo_sk#608)
               :- Project [ss_promo_sk#137, ss_quantity#139, ss_list_price#141, ss_sales_price#142, ss_coupon_amt#148, i_iten
               :  +- Join Inner, (ss_item_sk#131 = i_item_sk#564)
```

Spark Web UI

https://spark.apache.org/docs/latest/web-ui.html

When Data > Memory: How Spark Actually Works

Why “Too Big for Memory” Isn’t a Show-stopper

- Spark processes one partition at a time; it never needs the whole dataset in RAM.
- If memory is tight, operators spill; lineage allows safe recompute.
- Intermediates materialize only at shuffles or with cache/checkpoint.

Spark's Unified Memory—Who Uses What, When

- Unified pool split into Execution (joins/sort/agg) and Storage (cache).
- **Execution can evict Storage**; Storage can only borrow free space.
- Storage eviction is LRU for cached blocks.
- Key knobs: spark.executor.memory, spark.memory.fraction, off-heap.

How Spark Reads Large Data: One Partition at a Time

- **Tasks stream from sources; Storage memory uninvolved unless caching**.
- More/smaller partitions → higher parallelism, lower per-task memory.
- **Within-stage pipelining** (WholeStageCodegen) avoids intermediate materialization.
- Pipelining stops at **shuffles**, **cache/checkpoint**, and **stateful ops** (sort/agg/join/window).

Caching Rules that Actually Matter

- LRU applies **only when you cache()/persist()**.
- MEMORY_ONLY → evict = drop; recompute later.
- MEMORY_AND_DISK → blocks written to disk; read back when needed.
- *_SER reduces footprint at CPU cost.
- When to cache:
  - Reused ≥ 2 times (multiple actions/queries) or feeds expensive steps (post-shuffle, UDFs, window ops).
  - Trim first (filter/select) so the cached data is small.
  - Prefer MEMORY_AND_DISK(_SER) if it may not fit entirely in memory.
- After caching: optionally warm once with a cheap action if you need immediate reuse.
- **Unpersist** after last use: free memory; for imminent heavy stages, use a blocking unpersist.

What Happens When a Partition Is Too Big?

- A block **must fully fit in Storage** to be cached in memory.
- MEMORY_ONLY: block not cached; MEMORY_AND_DISK: stored on disk.
- Fixes: repartition (~128–256 MB), tune spark.sql.files.maxPartitionBytes, use *_SER.

When Operators Run Out of RAM: The Spill

- External sort and shuffle aggregation can spill intermediate data to local disk and merge it later.
- Hash-based joins require their build-side hash table to fit in memory and are not generally spill-safe.
- In the Stage UI, inspect Memory Bytes Spilled, Disk Bytes Spilled, Peak Execution Memory, and task duration.

Storage vs. Execution: Who Wins Under Pressure?

- Execution takes priority and can evict Storage blocks.
- Storage cannot evict Execution; caches may shrink during heavy joins/sorts.

Joins Under Memory Limits: Picking the Right Strategy

- Prefer **Broadcast Hash Join** when the build side fits comfortably in every executor.
- Prefer **Sort-Merge Join** for large equi-joins and memory-constrained workloads.
- Use **Shuffled Hash Join** only when each build-side shuffle partition is small enough to fit in memory. It is not generally spill-safe.
- Mitigate skew with AQE skew-join handling, pre-aggregation, heavy-key splitting, or salting.

Fault Tolerance without Replication: Lineage in Action

- Lineage recomputes lost partitions.
- Persistence reduces recompute cost; *_AND_DISK avoids recompute for cached blocks.
- Use checkpointing to truncate deep lineage.

Tuning Knobs that Move the Needle

- Partitioning: spark.sql.shuffle.partitions, spark.default.parallelism, repartition()/coalesce().
- Memory: executor memory, memoryOverhead, spark.memory.fraction, off-heap.
- Serialization: **KryoSerializer**, MEMORY_*_SER cache levels.
- I/O: spark.sql.files.maxPartitionBytes, openCostInBytes.
- Reminder: If you enable off-heap, raise executor.memoryOverhead accordingly.

See the Truth: Spark UI + explain(″extended″)

- UI: Spill bytes, Peak Exec Mem, skew indicators.
- Storage tab: cached size and location (mem/disk).
- Plans: explain("extended") reveals joins/shuffles and physical plan.

Best Practices & Common Mistakes

- Do
  - Cache small, reused intermediates; unpersist promptly:
    - Reuse criteria: referenced ≥2 times, across actions/jobs, or expensive to recompute.
    - Cache after trimming (filter/select/projection) to shrink footprint.
    - Pick level: small → MEMORY_ONLY(_SER); large/uncertain → MEMORY_AND_DISK(_SER).
    - Warm once if immediate reuse matters; then unpersist at last use (blocking if freeing space right before big joins/sorts).
  - Keep partitions even; fix skew early.
  - Prefer SMJ for large joins; broadcast only when it fits.
- Don't
  - `collect()` large data; `repartition(1)`.
  - Cache huge wide outputs "just in case".
  - Ignore spill metrics in the UI.

Cheat Sheet: Quick Decisions in the Heat of Battle

- Oversized partition → lower `files.maxPartitionBytes` or `repartition()`; for caching use `MEMORY_AND_DISK(_SER)`.
- Heavy spill → more partitions, more memory, or different join.
- Cache thrash → serialized / disk levels or stop caching.
- Join OOM risk → tune broadcast threshold; prefer SMJ; fix skew.

Summary: The Mental Model to Keep

- **Read**: partition-at-a-time streaming.
- **Cache**: blocks must fully fit; eviction via LRU on persist.
- **Execute**: operators spill incrementally and merge.
- **Priority**: Execution evicts Storage under pressure.